

Arborescence DevSecOps du projet Collector.shop

Documentation de l'implémentation : chaque fichier, son rôle,
pourquoi il existe, et ce qu'il apporte concrètement

Implémentation du cours « Pipeline DevSecOps GitOps avec Argo CD »
Go/Gin · SvelteKit · PostgreSQL · Kubernetes · Juin 2026

Sommaire

1. Vue d'ensemble : le flux complet
2. L'arborescence en un coup d'oeil
3. La CI backend — `.github/workflows/backend.yml`
4. La CI frontend — `.github/workflows/frontend.yml`
5. Le scan de secrets — `gitleaks.yml` et `dependabot.yml`
6. Les Dockerfiles durcis
7. `infra/k8s/base` — la description des services
8. `infra/k8s/overlays` — staging et prod
9. `infra/argocd` — le moteur GitOps
10. `infra/policies` — les garde-fous Kyverno
11. `infra/secrets` — Sealed Secrets
12. `infra/k8s/addons/rollouts` — le canary
13. Les modifications du code applicatif
14. Tableau récapitulatif fichier par fichier
15. Ce qu'il reste à faire (checklist)

1. Vue d'ensemble : le flux complet

Tout ce qui a été ajouté au repo sert un seul scénario : **un push de code arrive en staging sans aucune action manuelle, et rien de vulnérable ni de non signé ne peut passer**. La production, elle, exige deux décisions humaines : une PR de promotion et un clic Sync dans Argo CD.

```
push sur main (apps/backend/** ou apps/collector-front/**)
|
v
CI GitHub Actions
[1] tests + lint      (go test -race, golangci-lint / vitest, eslint)
[2] SAST              (gosec, Semgrep, SonarCloud)
[3] secrets           (gitLeaks, workflow separe, tous les push)
[4] SCA dependances  (govulncheck / trivy fs, npm audit, Dependabot)
    | tous verts
    v
[5] build multi-stage (hadolint, distroless nonroot)
[6] scan Trivy        (BLOQUANT CRITICAL/HIGH, AVANT le push)
[9] push GHCR         (tag sha-<commit>, jamais latest)
[7] SBOM syft         (SPDX, attachee a l'image)
[8] cosign keyless    (signature + attestation, journal Rekor)
    |
    v
job update-manifests : kustomize edit set image <digest>
commit dans infra/k8s/overlays/staging/ <- la CI n'a AUCUN acces au cluster
|
v
[11] Argo CD detecte le commit -> sync auto staging
[14] Kyverno verifie signature/non-root/tags a l'admission
[12] les secrets viennent de SealedSecrets commites
[15] Prometheus/Grafana observent le tout
    |
    v promotion = PR (copier le digest) + clic Sync
PRODUCTION
```

Les numéros [n] renvoient aux étapes du cours. Chaque section ci-dessous reprend les fichiers un par un avec le même schéma : ce qu'il contient, pourquoi il existe, un exemple concret de ce qu'il attrape ou permet, et les pièges à connaître.

2. L'arborescence en un coup d'oeil

```
collectorProject/
|-- .github/
|   |-- dependabot.yml          # PR automatiques de mise a jour (gomod, npm, actions, docker)
|   |-- workflows/
|       |-- backend.yml         # CI DevSecOps des services Go (remplace build.yml)
|       |-- frontend.yml        # CI DevSecOps du front SvelteKit
|       |-- gitleaks.yml        # scan de secrets, TOUS les push, tout l'historique
|-- apps/
|   |-- docker-compose.yml      # dev local (le front pointe sur Dockerfile.dev)
|   |-- backend/
|       |-- auth-service/       # Dockerfile durci (distroless static nonroot)
|       |-- catalog-service/    # idem + dossier uploads pre-cree pour nonroot
|       |-- notification-service/ # Dockerfile durci (service pas encore integre)
|       |-- price-tracker-service/ # idem
|   |-- collector-front/
|       |-- Dockerfile          # image de PRODUCTION (multi-stage, adapter-node)
|       |-- Dockerfile.dev      # serveur Vite + HMR pour docker-compose
|       |-- svelte.config.js     # adapter-node (au lieu d'adapter-auto)
|       |-- eslint.config.js     # lint bloquant, dette existante documentee
|-- docs/
|   |-- DEVSECOPS.md            # mapping des 15 etapes du cours -> le repo
|-- infra/                      # GitOps : l'etat desire du cluster
|   |-- README.md               # bootstrap k3d + Argo CD pas a pas
|   |-- argocd/
|       |-- bootstrap/root-app.yaml # App of Apps : LE manifest a appliquer a la main
|       |-- apps/                # 1 fichier = 1 application geree par Argo CD
|           |-- collector-staging.yaml (sync AUTO + selfHeal + prune)
|           |-- collector-prod.yaml   (sync MANUEL)
|           |-- sealed-secrets.yaml   (chiffrement des secrets)
|           |-- kyverno.yaml          (moteur de politiques)
|           |-- kyverno-policies.yaml (deploie infra/policies/)
|           |-- monitoring.yaml       (kube-prometheus-stack)
|           |-- argo-rollouts.yaml    (controller canary)
|-- k8s/
|   |-- base/                   # description unique des workloads (Kustomize)
|       |-- auth-service/       # deployment.yaml + service.yaml
|       |-- catalog-service/    # + pvc.yaml (photos uploadees)
|       |-- collector-front/    # deployment.yaml + service.yaml
|       |-- postgres/          # statefulset.yaml + service.yaml
|   |-- overlays/
|       |-- staging/            # namespace, ingress, digests epingles PAR LA CI
|       |-- prod/              # replicas/ressources sup., promotion par PR
|   |-- addons/rollouts/        # canary 25%/pause/100% pret a brancher
|-- policies/                   # ClusterPolicies Kyverno (verify-images, no-latest...)
|-- secrets/                     # template + procedure kubeseal (+ .gitignore)
```

3. La CI backend — .github/workflows/backend.yml

Se déclenche sur push/PR touchant **apps/backend/****. Sur PR, tout tourne (y compris le build et le scan d'image) mais rien n'est poussé ni signé. Six jobs :

Job lint-test — étape 1 du cours

CONTENU	gofmt (format), go vet (bugs probables), golangci-lint (méta-linter : errcheck, staticcheck, ineffassign, unused...), puis go test -race -cover . Matrix sur auth-service et catalog-service, couverture uploadée en artefact pour Sonar.
POURQUOI	Tout l'aval repose sur la confiance dans le code : déployer automatiquement n'a de sens que si on sait que ce qu'on déploie fonctionne. Le flag -race détecte les accès concurrents non protégés, la classe de bugs la plus dure à reproduire à la main.
EXEMPLE CONCRET	Si demain tu écris <code>var compteur int</code> incrémenté par deux goroutines sans mutex dans le catalog-service, le job échoue avec « WARNING: DATA RACE » et la stack des deux accès — au lieu d'un crash aléatoire en prod sous charge. errcheck, lui, refuse un <code>db.Save(&order)</code> dont l'erreur est ignorée : une commande « payée » qui n'a jamais été écrite en base.
A SAVOIR	golangci-lint a été exécuté en local avant d'être rendu bloquant : 0 issue sur les deux services, la CI ne deviendra donc pas rouge à cause de l'existant.

Job sonarcloud — qualité continue

CONTENU	Un scan SonarCloud unique pour tout apps/backend, alimenté par les deux fichiers coverage.out.
POURQUOI	L'ancien workflow lançait deux scans matrix avec le même projectKey : chaque scan écrasait le précédent dans SonarCloud, l'analyse affichée était celle du dernier service passé. Un seul scan = une vue fidèle et une seule Quality Gate.
A SAVOIR	Nécessite le secret SONAR_TOKEN (déjà en place). Les *_test.go sont exclus des sources et déclarés comme tests.

Job sast (gosec) — étape 2

CONTENU	gosec en sortie SARIF, bloquant (exit != 0 si finding), rapport visible dans l'onglet Security de GitHub, catégorie par service.
POURQUOI	Le développeur ne voit pas ses propres failles. gosec encode les pièges idiomatiques de Go sous forme de règles appliquées à chaque commit — exactement les classes de failles qu'un attaquant viserait en premier sur une marketplace avec comptes et transactions.
EXEMPLE CONCRET	Trois cas réels que gosec bloquerait dans ce code : G404 — générer un slug d'annonce MKT-xxx avec math/rand au lieu de crypto/rand (prédictible, un attaquant énumère les annonces) ; G101 — un JWT_SECRET par défaut en dur dans le code au lieu d'une variable d'environnement ; G304 — construire le chemin d'une photo à partir du nom de fichier envoyé par l'utilisateur sans le nettoyer (../etc/passwd dans /uploads).

Job sca (govulncheck) — étape 4

CONTENU	govulncheck (outil officiel Go) sur chaque module, bloquant.
---------	--

POURQUOI

80-90 % du code d'une appli moderne, ce sont les dépendances. govulncheck fait de l'**analyse d'appel** : il n'alerte que si la fonction vulnérable est réellement atteignable depuis ton code — quasiment zéro bruit, donc on peut le laisser bloquant sans fatigue d'alerte.

EXEMPLE CONCRET

Si une CVE sort demain sur le parsing multipart de gin (utilisé par l'upload de photos), le prochain push échoue avec la référence GO-20xx-xxxx, la fonction vulnérable atteinte et la version corrigée à prendre. Si la CVE touche une fonction de gin que le code n'appelle jamais : silence, pas de fausse alerte.

Job image — étapes 5, 6, 7, 8 et 9

CONTENU

Par service : **hadolint** (lint du Dockerfile) -> build buildx avec cache -> **Trivy** bloquant CRITICAL/HIGH sur l'image locale -> si push sur main : login GHCR, push taggé sha-<commit>, **SBOM syft** (SPDX json), **cosign sign + cosign attest** keyless, export du digest en artefact.

POURQUOI

L'ordre est la clé : l'image est scannée **avant** d'être poussée — une image avec une CVE critique n'atteint jamais le registre. La signature keyless (OIDC) prouve que l'image vient exactement de ce workflow sur ce repo : c'est ce que Kyverno vérifiera à l'admission. Le SBOM attaché répond en quelques secondes à « suis-je affecté ? » lors de la prochaine Log4Shell.

EXEMPLE CONCRET

Scénario de substitution d'image fermé par la signature : un token GHCR fuite, l'attaquant pousse une image piégée sous le même tag sha-abc1234. Sans signature, le cluster la déploierait docilement. Ici, cosign n'a jamais signé ce digest -> la policy verify-images la refuse à l'admission. C'est le scénario SolarWinds, à l'échelle du projet.

A SAVOIR

Permissions du job : packages:write (push GHCR avec le GITHUB_TOKEN éphémère — aucun secret longue durée), id-token:write (cosign keyless), security-events:write (SARIF Trivy). Sur PR, build + scan tournent mais sans push : on valide la PR au même niveau que main.

Job update-manifests — étapes 10/11 (le pont vers le GitOps)

CONTENU

Télécharge les digests exportés par les jobs image, fait kustomize edit set image image@sha256:... dans l'overlay staging, commit et push sur main.

POURQUOI

C'est LE principe GitOps : la CI ne parle jamais au cluster, elle écrit dans Git. Une CI compromise ne donne pas accès à la prod. L'historique de ce fichier kustomization.yaml est littéralement le journal des déploiements staging.

A SAVOIR

Deux protections anti-accident : un push fait avec GITHUB_TOKEN ne redéclenche pas de workflow (anti-boucle GitHub natif), et le groupe de concurrence **gitops-update-manifests** est partagé avec la CI frontend pour empêcher deux push simultanés (git pull --rebase en plus).

4. La CI frontend — .github/workflows/frontend.yml

Même squelette que le backend, adapté à l'écosystème npm/SvelteKit. Jobs : quality, tests, sast, sca, image, update-manifests.

Jobs quality + tests — étape 1

CONTENU

npm run lint (prettier --check + eslint, désormais **bloquant**), svelte-check (types), build de prod, puis vitest avec navigateur Playwright.

EXEMPLE CONCRET

Avant ce travail, « npm run lint » échouait sur 43 fichiers jamais formatés : tout le front a été reformaté avec le script du projet (npm run format) pour que le job parte au vert. Désormais, un import oublié ou un fichier non formaté bloque la PR au lieu de s'accumuler.

Job sast (Semgrep) — étape 2

CONTENU

Deux passes dans le conteneur officiel semgrep/semgrep : un scan complet --config auto exporté en SARIF (onglet Security), puis une passe **bloquante** limitée à la sévérité ERROR.

POURQUOI

gosec ne connaît pas TypeScript/Svelte. Semgrep apporte le registre communautaire : XSS, prototype pollution, open redirect... Bloquer seulement ERROR évite qu'une règle cosmétique bloque un déploiement, tout en gardant le rapport complet consultable.

EXEMPLE CONCRET

Si quelqu'un affiche la description d'une annonce avec {@html article.description} sans sanitisation, Semgrep le signale : un vendeur pourrait injecter du script et voler les sessions des acheteurs de la marketplace (XSS stocké — exactement l'exemple du cours).

Job sca — étape 4

CONTENU

Trivy fs sur le dossier du front (package-lock.json), bloquant CRITICAL/HIGH non corrigées + npm audit complet en informatif.

EXEMPLE CONCRET

Le front tire des centaines de packages transitifs. Cas type event-stream : un package populaire compromis publie une version malveillante — dès que l'advisory sort, le prochain push est bloqué avec le chemin de dépendance exact (qui dépend de quoi) et la version saine.

Job image + update-manifests — étapes 5 à 11

CONTENU

Identique au backend : hadolint, build, Trivy avant push, GHCR, SBOM, cosign, commit du digest en staging. Particularité : le build reçoit les URLs d'API en build-args.

A SAVOIR

Limitation assumée : les URLs d'API (VITE_*) sont compilées dans le bundle au moment du build (import.meta.env). L'image est donc buildée avec les hosts staging (auth.collector.staging.local / api.collector.staging.local). Une vraie prod front demande soit un build dédié, soit — mieux — un refactor vers \$env/dynamic/public de SvelteKit pour lire les URLs au runtime et restaurer le « build once, deploy many ». C'est le chantier n°1 de la doc.

5. Le scan de secrets et la veille automatique

`.github/workflows/gitleaks.yml`

CONTENU

Workflow dédié, déclenché sur **tous** les push et PR (pas de filtre de chemin), checkout avec `fetch-depth: 0` pour scanner **tout l'historique Git**, pas seulement le diff.

POURQUOI

C'était le point faible de l'ancien setup : gitleaks vivait dans la CI backend et ne tournait que si `apps/backend/**` changeait. Un mot de passe commité dans `infra/` ou dans le front passait sans scan. Or c'est l'erreur la plus fréquente et la plus immédiatement exploitable : un secret AWS poussé sur un repo public est exploité en moins de cinq minutes.

EXEMPLE CONCRET

Tu écris la procédure `Sealed Secrets`, et par réflexe tu commits `staging.unsealed.yaml` avec le vrai mot de passe Postgres. Triple filet : le `.gitignore` de `infra/secrets/` l'ignore, gitleaks bloque le push s'il passe quand même, et la Push Protection GitHub (à activer, gratuite) refuse le push côté serveur. Si malgré tout un secret atteint l'historique : suppression du fichier INSUFFISANTE — il faut révoquer/faire tourner le secret partout, l'historique est éternel.

`.github/dependabot.yml`

CONTENU

Veille automatique hebdomadaire : gomod (les 4 services), npm (collector-front, devDependencies groupées en une PR), github-actions, et docker (images de base des 5 Dockerfiles).

POURQUOI

Transforme la veille de sécurité en simple revue de PR : quand une dépendance est corrigée, la PR arrive toute seule, la CI complète (tests + scans) la valide, il ne reste qu'à merger.

EXEMPLE CONCRET

golang:1.25 publie un correctif, ou gin sort une version patchée : lundi matin, une PR « Bump gin-gonic/gin from v1.11.0 to v1.11.1 » t'attend, déjà passée au travers de toute la chaîne (tests, gosec, govulncheck, Trivy).

6. Les Dockerfiles durcis — étape 5

apps/backend/*/Dockerfile (auth, catalog, notification, price-tracker)

CONTENU

Stage 1 : golang:1.25 (ou 1.23) compile un binaire statique (CGO_ENABLED=0, -trimpath, -ldflags -s -w). Stage 2 : **gcr.io/distroless/static-debian12:nonroot** — uniquement le binaire, les CA certs et tzdata. USER nonroot, ENTRYPOINT exec-form.

POURQUOI

La sécurité d'une image se mesure à sa surface d'attaque. distroless static n'a NI shell, NI gestionnaire de paquets, NI curl : même compromis, le conteneur n'offre presque aucun outil de post-exploitation. Bénéfice annexe : le scan Trivy est quasi vide (pas de paquets Debian inutilisés qui noient les vraies alertes) et l'image fait quelques Mo.

EXEMPLE CONCRET

Avant/après concret : l'ancienne base de l'auth-service était distroless/base-debian12 (avec libc, openssl...) — chaque CVE d'openssl déclenchait une alerte Trivy alors que le binaire Go statique ne l'utilise même pas. Avec static-debian12, ces alertes disparaissent ; celles qui restent sont de vraies alertes sur TES dépendances.

A SAVOIR

Cas particulier catalog-service : il écrit les photos dans ./uploads. Le dossier est pré-crée dans le builder et copié avec --chown=65532:65532 (l'uid de nonroot) — sinon MkdirAll échouerait, /app appartenant à root. En compose, le volume nommé hérite de ces permissions ; en Kubernetes c'est fsGroup: 65532 qui s'en charge. Les HEALTHCHECK wget des anciens Dockerfiles ont été retirés : pas de wget dans distroless, les probes Kubernetes les remplacent.

apps/collector-front/Dockerfile + Dockerfile.dev

CONTENU

Dockerfile = production : stage builder (npm ci, npm run build avec adapter-node, npm prune --omit=dev), stage final node:22.12-alpine qui ne contient que build/, package.json et les node_modules de prod ; USER node, port 3000, CMD node build. Dockerfile.dev = l'ancien serveur Vite+HMR sur 5173, utilisé par docker-compose.

POURQUOI

L'ancien Dockerfile unique lançait « npm run dev » : un serveur de développement (non optimisé, HMR exposé, toutes les devDependencies embarquées) ne doit jamais être l'artefact déployé. Séparer les deux permet d'avoir une image de prod propre SANS casser le workflow de dev local — docker-compose pointe explicitement sur Dockerfile.dev, rien ne change au quotidien.

EXEMPLE CONCRET

Les ARG VITE_* du Dockerfile de prod sont le mécanisme par lequel la CI fixe les URLs d'API du bundle. Pour tester l'image de prod en local : docker build -t front-prod apps/collector-front && docker run -p 3000:3000 front-prod.

7. infra/k8s/base — la description des services (étape 10)

La base décrit chaque service **une seule fois**, sans namespace ni tag d'image : ce sont les overlays qui spécialisent. `kustomization.yaml` liste les 9 ressources et appose le label commun `app.kubernetes.io/part-of: collector`.

`auth-service/deployment.yaml` · `catalog-service/deployment.yaml`

CONTENU

Deployment 1 replica avec : probes readiness/liveness sur GET /health ; envFrom configmap collector-config (DB_HOST, DB_PORT, DB_NAME, DB_USER, FRONTEND_ORIGIN) ; JWT_SECRET et DB_PASSWORD lus dans le Secret collector-secrets ; requests/limits CPU et mémoire ; et un securityContext complet.

POURQUOI

Chaque ligne du securityContext ferme une porte : **runAsNonRoot** + **runAsUser 65532** (l'évasion vers le noeud passe presque toujours par root), **readOnlyRootFilesystem** (un attaquant ne peut pas déposer de binaire), **capabilities drop ALL** et **allowPrivilegeEscalation: false** (pas de remontée de privilèges), **automountServiceAccountToken: false** (pas de token API Kubernetes offert à un éventuel code compromis), **seccompProfile RuntimeDefault** (syscalls filtrés).

EXEMPLE CONCRET

La readiness probe n'est pas cosmétique : pendant un rolling update, un pod qui démarre mais dont la connexion Postgres échoue ne devient jamais Ready -> il ne reçoit AUCUN trafic et le rollout se fige au lieu de basculer les utilisateurs vers un service cassé. Sans probe, Kubernetes bascule dès que le process tourne.

A SAVOIR

catalog-service en plus : volume PVC monté sur /app/uploads + fsGroup 65532 (volume inscriptible par nonroot malgré le readOnlyRootFilesystem, qui ne s'applique qu'au système de fichiers de l'image). Et **replicas: 1 obligatoire** tant que le volume est en ReadWriteOnce — deux pods sur deux noeuds ne peuvent pas monter le même volume RWO. Pour scaler : stockage objet (S3/minio) ou volume RWX.

`collector-front/deployment.yaml`

CONTENU

Même durcissement, port 3000, runAsUser 1000 (l'utilisateur node de l'image alpine), probes sur GET /.

A SAVOIR

runAsUser est **numérique** exprès : l'image déclare USER node (un nom) et la kubelet ne peut pas vérifier runAsNonRoot face à un nom -> CreateContainerConfigError. Le chiffre 1000 lève l'ambiguïté.

`postgres/statefulset.yaml` · `postgres/service.yaml`

CONTENU

StatefulSet 1 replica, postgres:17-alpine, volumeClaimTemplate 2 Gi, PGDATA dans un sous-dossier, fsGroup 70 (uid postgres d'alpine), probes pg_isready, mot de passe lu dans collector-secrets (clé DB_PASSWORD, la même que celle des services). Service ClusterIP « postgres » -> DB_HOST=postgres dans la configmap.

POURQUOI

StatefulSet plutôt que Deployment : identité stable + PVC lié au pod, le pattern pour les bases de données. PGDATA en sous-dossier car le point de montage lui-même appartient à root et initdb refuse un répertoire non vide.

A SAVOIR

Une base mutualisée dans le cluster convient au projet de cours ; en production réelle on prendrait une base managée ou un opérateur (CloudNativePG) pour les sauvegardes et la HA.

8. infra/k8s/overlays — staging et prod (étape 10)

overlays/staging/kustomization.yaml

CONTENU

namespace collector-staging, la base + ingress.yaml, configMapGenerator de collector-config (FRONTEND_ORIGIN staging) et le bloc **images:** — c'est LE fichier que la CI modifie : chaque déploiement remplace le tag par un digest sha256 immuable.

POURQUOI

git log sur ce fichier = le journal des déploiements staging : qui a déployé quoi, quand, avec quel diff. Le digest (et pas un tag) garantit qu'un tag re-poussé frauduleusement ne changerait rien : le digest est cryptographiquement lié au contenu.

EXEMPLE CONCRET

Rollback en un revert : le déploiement de 14h32 casse la page /marche -> git revert du commit « ci(gitops): deploy backend a1b2c3d to staging » -> Argo CD resynchronise l'ancien digest en quelques secondes. Pas de kubectl, pas de stress.

A SAVOIR

Le tag initial « bootstrap » est un placeholder volontairement impossible à puller : tant que la CI n'a pas tourné une fois, le pod reste en ImagePullBackOff au lieu de tirer silencieusement un latest quelconque. La ligne sealed-collector-secrets.yaml est commentée : à décommenter après génération du SealedSecret. Le configMapGenerator suffixe le nom d'un hash : changer la config redéploie automatiquement les pods qui la consomment.

overlays/prod/kustomization.yaml

CONTENU

namespace collector-prod, FRONTEND_ORIGIN prod, et des patches JSON : replicas 2 pour auth-service et collector-front, limites mémoire doublées. catalog-service reste à 1 replica (volume RWO). Les images sont épinglées **à la main**.

POURQUOI

Personne — pas même la CI — ne déploie en prod sans revue : la promotion consiste à copier le digest validé en staging dans ce fichier via une PR, puis à cliquer Sync dans Argo CD. Deux validations humaines, toutes deux tracées (la PR et l'événement de sync).

overlays/*/ingress.yaml

CONTENU

Routage par hôte (Traefik, inclus dans k3s/k3d) : collector[.staging].local -> front, auth.collector[.staging].local -> auth-service, api.collector[.staging].local -> catalog-service. Les Services exposent le port 80 vers le port conteneur.

A SAVOIR

En local, ajouter les 6 hôtes dans le fichier hosts (127.0.0.1) — commandes dans infra/README.md. Le choix host-based (plutôt que path-based) évite des règles de réécriture d'URL : les services Go n'ont pas de préfixe /api dans leurs routes.

9. infra/argocd — le moteur GitOps (étape 11)

bootstrap/root-app.yaml — le pattern App of Apps

CONTENU

Une Application Argo CD qui pointe sur... le dossier infra/argocd/apps du même repo. C'est le SEUL manifest appliqué à la main (kubectl apply -f) de toute l'infrastructure.

POURQUOI

Effet domino : Argo CD lit ce dossier et crée toutes les autres applications, qui installent à leur tour sealed-secrets, kyverno, le monitoring et les deux environnements. Ajouter un composant d'infra = committer un YAML de plus dans apps/. Après un sinistre, le cluster entier se reconstruit depuis Git avec cette seule commande (+ la clé sealed-secrets sauvegardée).

Le dossier apps/ — une application par fichier

collector-staging.yaml	sync AUTOMATED + prune + selfHeal + CreateNamespace
collector-prod.yaml	sync MANUEL (le clic Sync = validation de mise en prod)
sealed-secrets.yaml	chart Helm bitnami-labs, kube-system, fullnameOverride=sealed-secrets-controller (default kubeseal)
kyverno.yaml	chart Helm 3.x, ServerSideApply (CRDs volumineuses)
kyverno-policies.yaml	pointe sur infra/policies/ du repo
monitoring.yaml	kube-prometheus-stack, retention 7j, serviceMonitorSelectorNilUsesHelmValues=false
argo-rollouts.yaml	controller canary/blue-green (utilise par addons/rollouts)

POURQUOI

Les trois mécanismes de collector-staging font tout le sel du GitOps : **sync auto** (un commit est appliqué en ~3 min, instantané avec un webhook), **selfHeal** (un kubectl edit sauvage est détecté comme drift et réaligné sur Git — la prod ne peut plus diverger silencieusement), **prune** (une ressource supprimée de Git est supprimée du cluster, zéro orphelin).

EXEMPLE CONCRET

Test à faire en démo : kubectl scale deploy auth-service --replicas=5 -n collector-staging. Quelques secondes plus tard, Argo CD remet 1 replica et marque l'événement dans son historique. La modification « temporaire » non documentée — la dette opérationnelle classique — est devenue impossible.

A SAVOIR

Les targetRevision des charts Helm utilisent des plages (2.x, 3.x, *) pour simplifier le bootstrap : à épingler sur des versions précises après la première installation pour des upgrades maîtrisés. Le monitoring expose Grafana via kubectl port-forward -n monitoring svc/monitoring-grafana 3001:80.

10. infra/policies — les garde-fous Kyverno (étape 14)

Quatre ClusterPolicies, toutes limitées aux namespaces collector-staging / collector-prod, toutes en **validationFailureAction: Audit** pour commencer : les violations apparaissent dans les PolicyReports (kubectl get policyreport -A) sans bloquer. Une fois les faux positifs purgés, passer à Enforce — c'est le commentaire en tête de chaque fichier.

verify-images.yaml — la clé de voûte

CONTENU

Règle verifyImages : toute image ghcr.io/jtr220/collector/* doit porter une signature cosign keyless émise par un workflow GitHub Actions du repo JTR220/collector (subject + issuer OIDC vérifiés contre le journal de transparence Rekor). mutateDigest résout les tags en digests.

POURQUOI

La signature de l'étape 8 ne vaut que si quelqu'un la vérifie. Avec cette policy en Enforce, « ce qui n'est pas né dans ta CI ne tourne pas » devient une règle d'admission, pas une convention : kubectl run pirate --image=ghcr.io/jtr220/collector/auth-service@sha256:evil... est rejeté AVANT la création du pod, même lancé par un admin.

disallow-latest-tag.yaml

CONTENU

Deux règles : une image doit avoir un tag ou un digest explicite, et :latest est interdit.

EXEMPLE CONCRET

Avec latest, deux pods du même deployment peuvent exécuter deux images différentes selon le moment du pull, et un rollback devient une devinette. La policy rend la règle d'or du registre (étape 9) inviolable, y compris pour un manifest écrit à la main un vendredi soir.

require-run-as-nonroot.yaml · require-resources.yaml

CONTENU

runAsNonRoot obligatoire (pod ou conteneur) ; requests CPU/mémoire + limite mémoire obligatoires.

POURQUOI

Défense en profondeur : les Dockerfiles sont déjà nonroot et les Deployments ont déjà leurs ressources — mais un chart Helm tiers laxiste ou un manifest expérimental, eux, n'auront pas ces garanties. Sans limits, un pod qui fuit la mémoire OOM-kill ses voisins de noeud ; sans requests, le scheduler place à l'aveugle.

11. infra/secrets — Sealed Secrets (étape 12)

README.md · collector-secrets.template.yaml · .gitignore

CONTENU

La procédure complète kubeseal, un template du Secret collector-secrets (JWT_SECRET, DB_PASSWORD) avec des placeholders CHANGE-ME, et un .gitignore qui exclut les fichiers de travail *.unsealed.yaml.

POURQUOI

Le paradoxe GitOps : tout l'état est dans Git, mais un Secret Kubernetes n'est que du base64 — de l'encodage, pas du chiffrement, le committer équivaut à le publier. kubeseal chiffre avec la clé publique du cluster : le SealedSecret produit est committable sans risque, seul le controller détient la clé privée.

EXEMPLE CONCRET

Flux complet : `cp template staging.unsealed.yaml -> remplir avec openssl rand -base64 32 -> kubeseal < staging.unsealed.yaml > overlays/staging/sealed-collector-secrets.yaml -> rm le fichier en clair -> décommenter la ressource dans le kustomization -> commit. Au sync suivant, le controller crée le Secret que consomment les Deployments et Postgres.`

A SAVOIR

Deux pièges : un SealedSecret est chiffré pour UN namespace et UN nom précis (en générer un par environnement, avec des valeurs différentes) ; et il faut **sauvegarder la clé privée du controller hors du repo** (commande dans le README) — sinon, après réinstallation du cluster, tous les SealedSecrets commités sont indéchiffrables.

12. infra/k8s/addons/rollouts — le canary (étape 13)

[rollout-catalog-service.yaml](#) · [README.md](#)

CONTENU

Un Rollout Argo Rollouts en stratégie canary à deux paliers (25 % -> pause manuelle -> 100 %) qui référence le Deployment existant via `workloadRef` (pas de duplication de manifest), plus un squelette commenté d'AnalysisTemplate Prometheus (promotion auto si < 5 % d'erreurs 5xx).

POURQUOI

Même avec tous les scans du monde, certains problèmes n'apparaissent qu'en production : fuite mémoire sous charge réelle, latence dégradée. Le canary limite le rayon d'explosion : une régression touche 25 % des requêtes pendant la pause, pas 100 % des transactions d'un coup.

A SAVOIR

Volontairement NON branché aux overlays : il nécessite le controller argo-rollouts. L'activation tient en une ligne de kustomization (voir le README du dossier). L'analyse automatique attend l'instrumentation Prometheus des services Go — c'est pour ça que le template est commenté.

13. Les modifications du code applicatif

- **svelte.config.js** : adapter-auto -> **adapter-node** (dépendance ajoutée au package.json). adapter-auto ne sait pas produire un serveur autonome pour Docker/Kubernetes ; adapter-node génère build/ lancé par « node build ». Le dev local (vite dev) n'utilise pas l'adapter : aucun impact.
- **eslint.config.js** : 3 règles svelte désactivées (no-navigation-without-resolve, require-each-key, prefer-svelte-reactivity) et no-unused-vars en warning — 110 occurrences préexistantes dans le code. Le choix : rendre le lint bloquant MAINTENANT pour tout nouveau code, et résorber la dette règle par règle au fil des refactors (commentaire dans le fichier).
- **Tout le front reformaté** avec npm run format (prettier) : 43 fichiers, diff purement cosmétique, vérifié par un rebuild complet.
- **apps/docker-compose.yml** : le service collector-front pointe sur Dockerfile.dev — le workflow de dev local est inchangé (Vite + HMR sur 5173).
- **docs/DEVSECOPS.md** : le mapping des 15 étapes du cours vers le repo, les garde-fous notables, les limites connues et la checklist des actions manuelles — le point d'entrée pour une soutenance.

14. Tableau récapitulatif

Fichier / dossier	Rôle	Étape	Risque couvert
.github/workflows/backend.yml	CI DevSecOps des services Go	1-11	Régression, faille, image vérolée déployées
.github/workflows/frontend.yml	CI DevSecOps du front	1-11	XSS, CVE npm, image dev en prod
.github/workflows/gitleaks.yml	Scan de secrets global	3	Secret commité = compromission totale
.github/dependabot.yml	PR de mise à jour auto	4	CVE connues non patchées
apps/*/Dockerfile	Images distroless non-root	5	Surface d'attaque, post-exploitation
infra/k8s/base/	Description unique des workloads	10	YAML dupliqués qui divergent
infra/k8s/overlays/staging/	Env. auto-déployé (digests CI)	10	Déploiements opaques, rollback difficile
infra/k8s/overlays/prod/	Env. promu par PR + Sync manuel	10	Mise en prod sans revue
infra/argocd/bootstrap/root-app.yaml	App of Apps, bootstrap unique	11	Infra non reproductible
infra/argocd/apps/	Toutes les applications du cluster	11-15	Credentials cluster dans la CI, drift
infra/policies/	Admission control Kyverno	14	Pipeline contournable par kubectl
infra/secrets/	Procédure Sealed Secrets	12	Secrets dans Git ou non reproductibles
infra/k8s/addons/rollouts/	Canary prêt à brancher	13	Régression sur 100 % du trafic
docs/DEVSECOPS.md	Mapping cours -> repo + checklist	-	Perte de la vue d'ensemble

15. Ce qu'il reste à faire (checklist)

Côté GitHub (5 minutes)

- Activer **Secret Scanning** + **Push Protection** (Settings -> Code security and analysis).
- **Protéger la branche main** : PR obligatoire + checks requis (gitleaks, lint-test, sast, sca, image). Sans ça, les jobs « bloquants » restent des conventions contournables par un push direct.
- Après le premier push d'images : vérifier la visibilité des packages GHCR (public, ou créer un imagePullSecret en lecture seule pour le cluster — jamais le token d'écriture).

Côté cluster (1 à 2 week-ends — Phase 1 du cours)

```
k3d cluster create collector --port "80:80@loadbalancer"
kubectl create namespace argocd
helm repo add argo https://argoproj.github.io/argo-helm
helm install argocd argo/argo-cd -n argocd
kubectl apply -f infra/argocd/bootstrap/root-app.yaml # <- tout découle de la
# puis : SealedSecrets (infra/secrets/README.md) + entrees dans le fichier hosts
```

Au fil de l'eau

- Passer les politiques Kyverno d'**Audit** à **Enforce** une fois les PolicyReports purgés.
- Écrire de vrais **tests unitaires backend** : le go test -race de la CI passe, mais une barrière de qualité sans tests est en partie creuse.
- **Instrumenter les services Go** (prometheus/client_golang : latence, erreurs, métriques métier) -> ServiceMonitors -> analyse canary automatique de l'étape 13.
- Refactorer le front vers **\$env/dynamic/public** (URLs d'API lues au runtime) pour restaurer le « build once, deploy many » et déployer la même image front en staging et en prod.
- Brancher notification-service et price-tracker-service (CI + manifests) quand ils seront intégrés.
- Épingler les versions des charts Helm (2.x / 3.x / * -> versions précises) après le bootstrap.

Le mot de la fin du cours s'applique tel quel à ce repo : cette arborescence n'est pas une liste d'outils empilés, c'est une chaîne de confiance. Les tests garantissent que le code fonctionne, les scans qu'il est sain, la signature qu'il n'a pas été altéré, Argo CD qu'il est déployé fidèlement, Kyverno que rien ne contourne le tout, et l'observabilité que tu le vois. Retirer un fichier ne retire pas une fonctionnalité : il casse une garantie sur laquelle tout l'aval reposait.